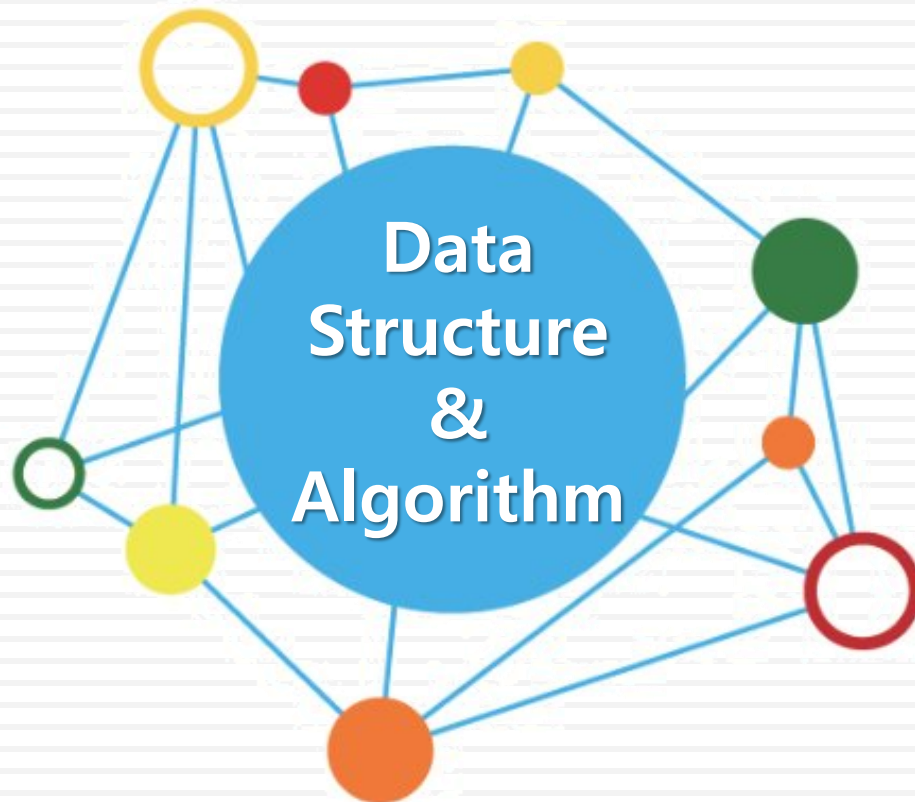


데이터 구조론



유길현

스택(Stack)

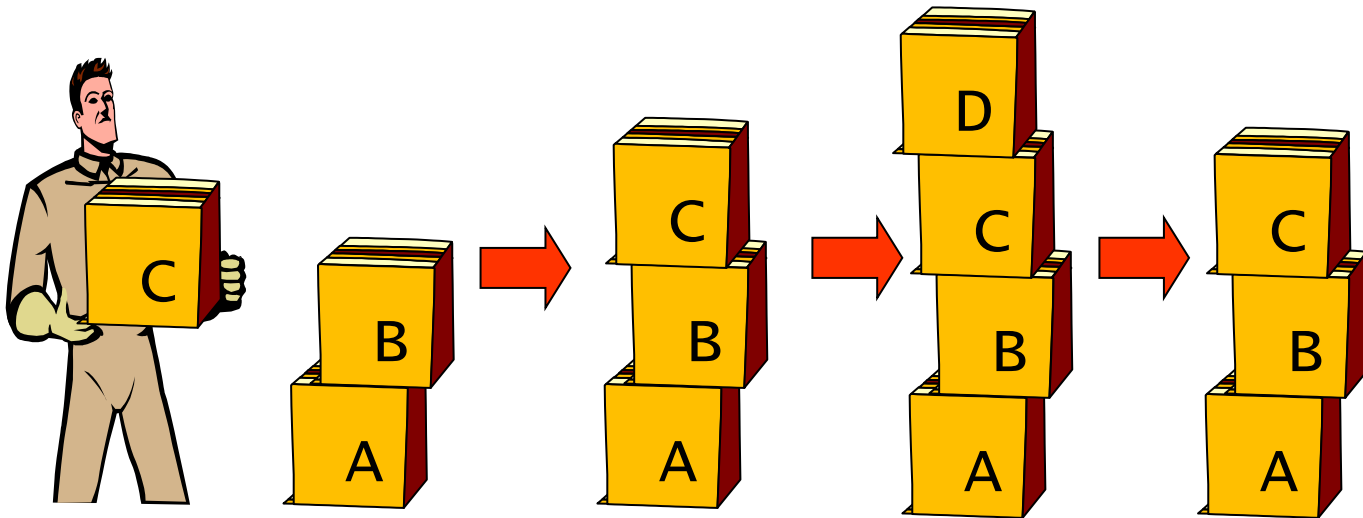
❖ 스택(stack)이란?

- 자료를 차곡차곡 쌓아 올린 형태의 데이터구조



❖ 스택(Stack)의 특징

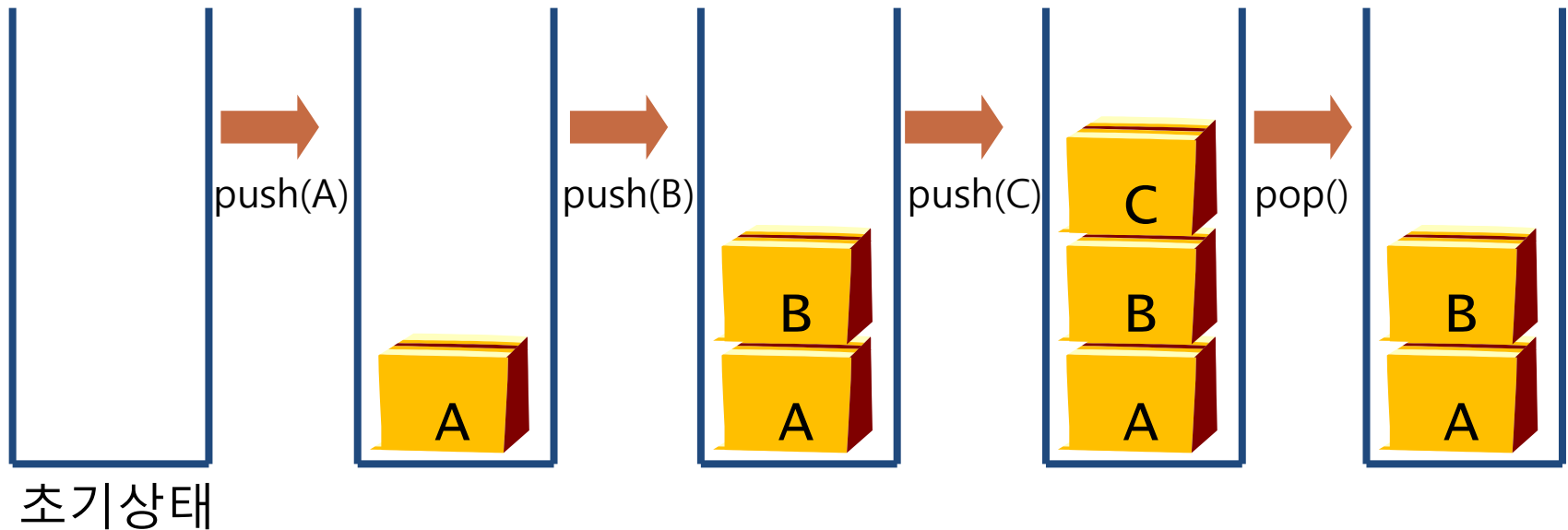
- 스택에 저장된 원소는 top으로 정한 곳에서만 접근 가능
 - top의 위치에서만 원소를 삽입하므로, 먼저 삽입한 원소는 밑에 쌓이고, 나중에 삽입한 원소는 위에 쌓이는 구조
 - 마지막에 삽입(**Last-In**)한 원소는 맨 위에 쌓여 있다가 가장 먼저 나감(**FirstOut**)됨 ➡ **후입선출 구조 (LIFO, Last-In-First-Out)**



스택의 개념과 구조

❖ 스택의 원소 삽입/삭제 연산

- 스택에서의 삽입 연산 : push
- 스택에서의 삭제 연산 : pop



ADT(Abstract Data Type)

스택의 추상 데이터형

ADT Stack

데이터 : 0개 이상의 원소를 가진 유한 선형 리스트

연산 :

$S \in \text{Stack}; \text{item} \in \text{Element};$

// 공백 스택 S를 생성 하는 연산

CreateStack(S) ::= create an empty Stack S;

// 스택 S가 공백인지 확인하는 연산

isEmpty(S) ::= if(S is empty) then return true
 else return false;

// 스택 S의 top에 item(원소)을 삽입하는 연산

push(S, item) ::= insert item onto the top of Stack S;

// 스택 S의 top에 있는 item(원소)을 삭제하는 연산

pop(S) ::= if(isEmpty(S)) then return error
 else delete and return the top item of Stack S;

// 스택 S의 top에 있는 item(원소)을 반환하는 연산

peek(S) ::= if(isEmpty(S)) then return error
 else return the top item of Stack S;

End Stack

❖ 스택의 push() 알고리즘

① $top \leftarrow top + 1;$

- 스택 S에서 top이 마지막 자료를 가리키고 있으므로 그 위에 자료를 삽입하려면 먼저 top의 위치를 하나 증가
- 만약 이때 top의 위치가 스택의 크기(stack_SIZE)보다 크다면 오버플로우(overflow)상태가 되므로 삽입 연산을 수행하지 못하고 연산 종료

② $S(top) \leftarrow x;$

- 오버플로우 상태가 아니라면 스택의 top이 가르키는 위치에 x 삽입

알고리즘

스택의 원소 삽입

```
push(S, x)
```

```
  ①  $top \leftarrow top + 1;$ 
```

```
    if ( $top > stack\_SIZE$ ) then overflow;
```

```
    else
```

```
      ②  $S(top) \leftarrow x;$ 
```

```
end push()
```

❖ 스택의 pop() 알고리즘

① return S(top);

– 공백 스택이 아니면 top에 있는 원소를 삭제하고 반환

② top $\leftarrow$ top - 1;

– 스택의 마지막 원소가 삭제되면 그 아래 원소, 즉 스택에 남아 있는 원소 중에서 가장 위에 있는 원소가 top이 되어야 하므로 top 위치를 하나 감소

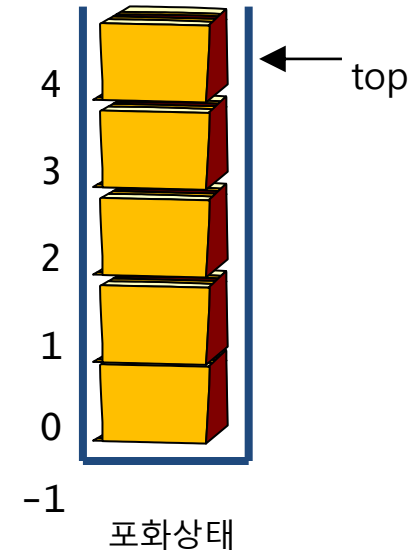
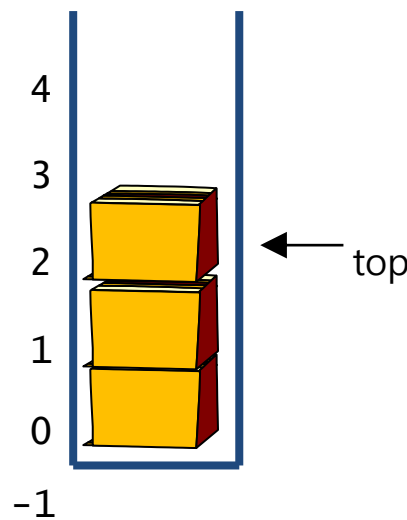
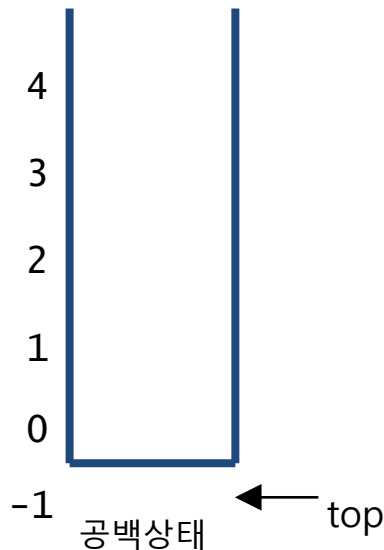
알고리즘

스택의 원소 삭제

```
pop(S)
  if(top == 0) then underflow;
  else{
    ① return S(top);
    ② top  $\leftarrow$  top - 1;
  }
end pop()
```

❖ 순차 자료 구조를 이용한 스택 구현

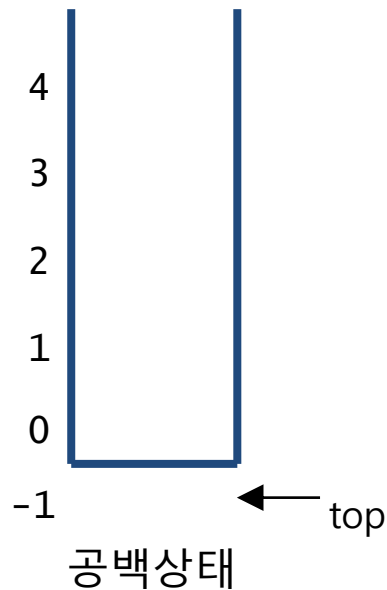
- 순차 자료구조인 1차원 배열을 이용하여 구현
 - 1차원 배열 `stack[ ]`
 - 스택에서 가장 최근에 입력되었던 자료를 가리키는 `top` 변수
 - 가장 먼저 들어온 요소는 `stack[0]`에, 가장 최근에 들어온 요소는 `stack[top]`에 저장
 - 스택이 공백이면 `top = -1` (초기값)
 - 스택이 포화상태 이면 `top = n-1`



❖ 공백 과 포화 상태의 연산

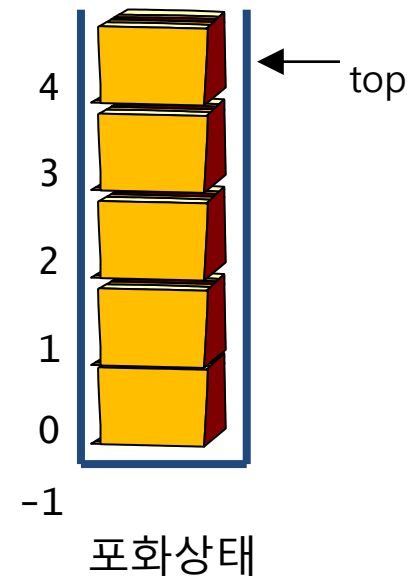
```
is_empty(S)
```

```
if top = -1  
  then return TRUE  
  else return FALSE
```



```
is_full(S)
```

```
if top = (MAX_STACK_SIZE-1)  
  then return TRUE  
  else return FALSE
```



❖ push 연산

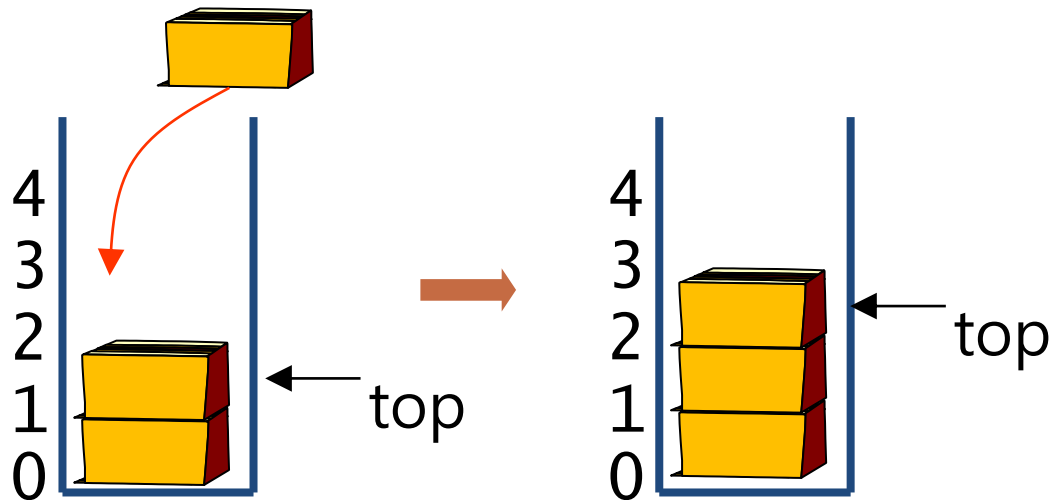
```
push(S, x)
```

```
if is_full(S)
```

```
    then error "overflow"
```

```
    else top ← top + 1
```

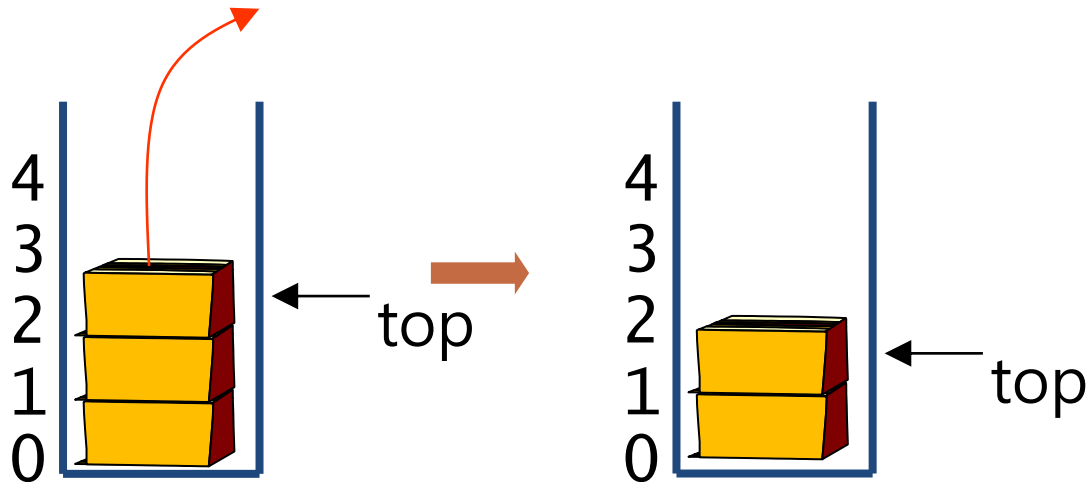
```
        stack[top] ← x
```



❖ pop 연산

```
pop(S, x)
```

```
if is_empty(S)  
  then error "underflow"  
  else e ← stack[top]  
       top ← top - 1  
       return e
```



❖ 예제 (1/5)

```
#define _CRT_SECURE_NO_WARNINGS
#include "stdio.h"
#include "stdlib.h"
#define STACK_SIZE 100

typedef int element;    // 스택 원소(element)의 자료형을 int로 정의

element stack[STACK_SIZE];    // 1차원 배열 스택 선언
int top = -1;                // top 초기화

// 스택이 공백 상태인지 확인하는 연산
int isEmpty()
{
    if (top == -1)
        return 1;
    else
        return 0;
}
```

❖ 예제 (2/5)

```
int isFull()                // 스택이 포화 상태인지 확인하는 연산
{
    if (top == STACK_SIZE - 1)
        return 1;
    else
        return 0;
}

void push(element item)     // 스택의 top에 원소를 삽입하는 연산
{
    if (isFull())           // 스택이 포화 상태인 경우
    {
        printf("\n\n Stack is FULL! \n");
        return;
    }
    else
        stack[++top] = item; // top을 증가시킨 후 현재 top에 원소 삽입
}
```

❖ 예제 (3/5)

```
element pop()                // 스택의 top에서 원소를 삭제하는 연산
{
    if (isEmpty())           // 스택이 공백 상태인 경우
    {
        printf("\n\n Stack is Empty!!\n");
        return 0;
    }
    else
        return stack[top--]; // 현재 top의 원소를 삭제한 후 top 감소
}
```

❖ 예제 (4/5)

```
element peek()                // 스택의 top 원소를 검색하는 연산
{
    if (isEmpty())            // 스택이 공백 상태인 경우
    {
        printf("\n\n Stack is Empty !\n");
        exit(1);
    }
    else
        return stack[top];    // 현재 top의 원소 확인
}

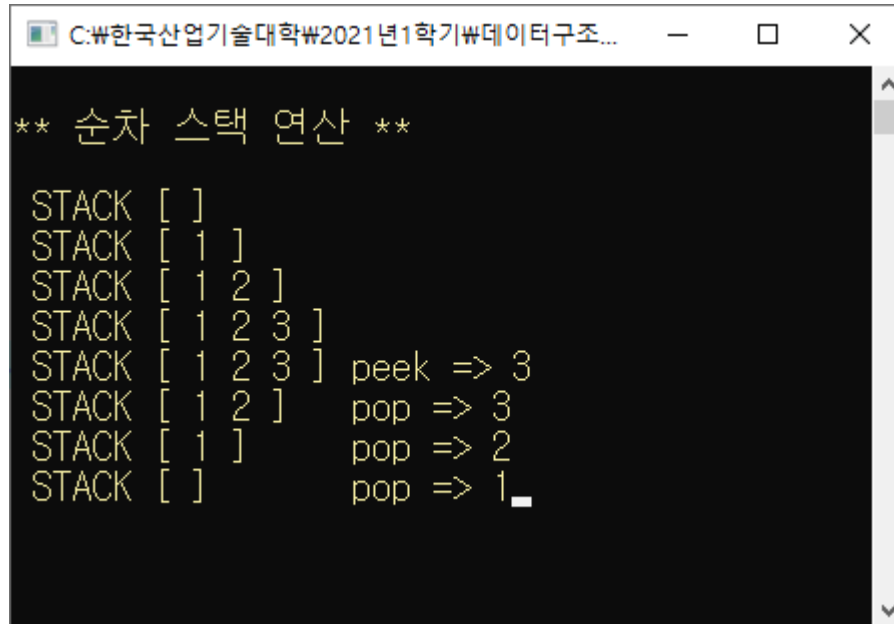
void printStack()              // 스택의 원소를 출력하는 연산
{
    int i;
    printf("\n STACK [ ");
    for (i = 0; i <= top; i++)
        printf("%d ", stack[i]);
    printf("] ");
}
```

❖ 예제 (5/5)

```
void main()
{
    int i;
    element item;
    printf("\n** 순차 스택 연산 **\n");
    printStack();
    for (i = 1; i <= 3; i++){
        push(i);
        printStack();
    }
    item = peek();
    printStack();           // 현재 top의 원소 출력
    printf("peek => %d", item);
    for (i = 0; i < 3; i++) {
        item = pop();
        printStack();       // 현재 top의 원소 출력
        printf("\t pop => %d", item);
    }
    getchar();
}
```


❖ 예제 : 교재 239p ~241p

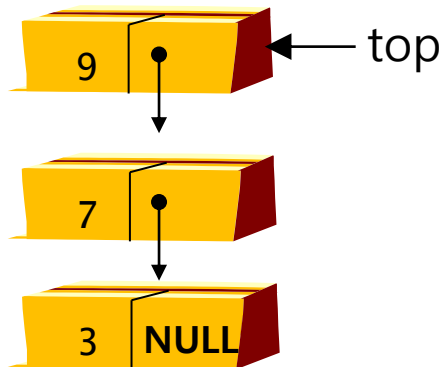
■ 실행 결과



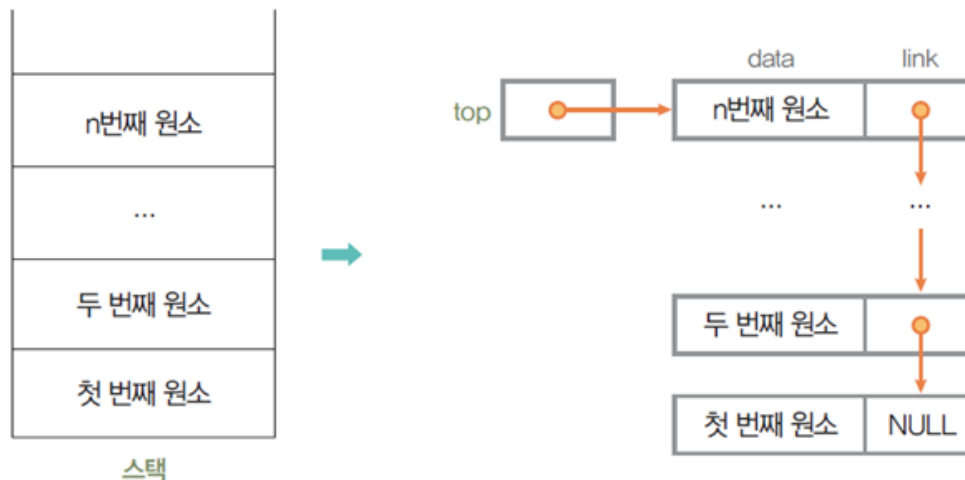
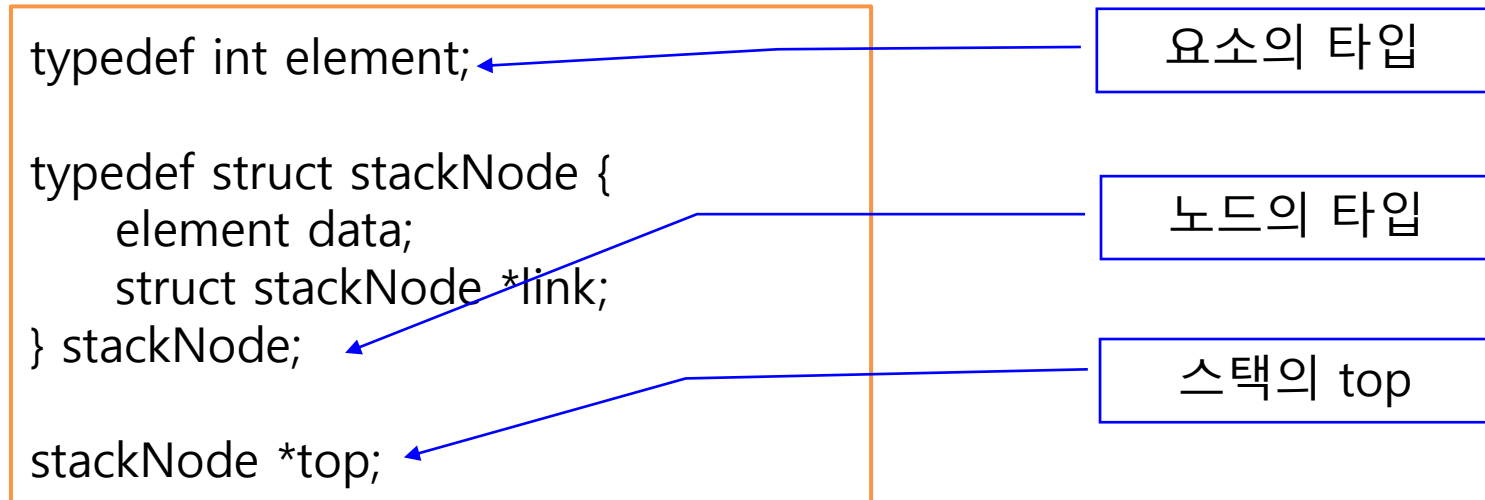
```
** 순차 스택 연산 **  
  
STACK [ ]  
STACK [ 1 ]  
STACK [ 1 2 ]  
STACK [ 1 2 3 ]  
STACK [ 1 2 3 ] peek => 3  
STACK [ 1 2 ] pop => 3  
STACK [ 1 ] pop => 2  
STACK [ ] pop => 1
```

❖ 단순 연결 리스트를 이용하여 구현

- 스택의 원소 : 단순 리스트의 노드
 - 스택 원소의 순서 : 노드의 링크 포인터로 연결
 - push : 리스트의 마지막 노드 삽입
 - pop : 리스트의 마지막 노드 삭제
- 변수 top : 단순 연결 리스트의 마지막 노드를 가리키는 포인터 변수
 - 초기 상태 : top = null
- 장점 : 크기가 제한되지 않음
- 단점 : 구현이 복잡하고 삽입/삭제 시간이 오래 걸림



❖ 연결 리스트 스택의 정의



단순 연결 리스트를 이용한 연결스택

❖ 연결 리스트 스택의 push 연산

```
// 스택의 top에 원소를 삽입하는 연산
void push(element item)
{
    stackNode* temp = (stackNode *)malloc(sizeof(stackNode));
    temp->data = item;
    temp->link = top;    // 삽입 노드를 top의 위에 연결
    top = temp;          // top 위치를 삽입 노드로 이동
}
```

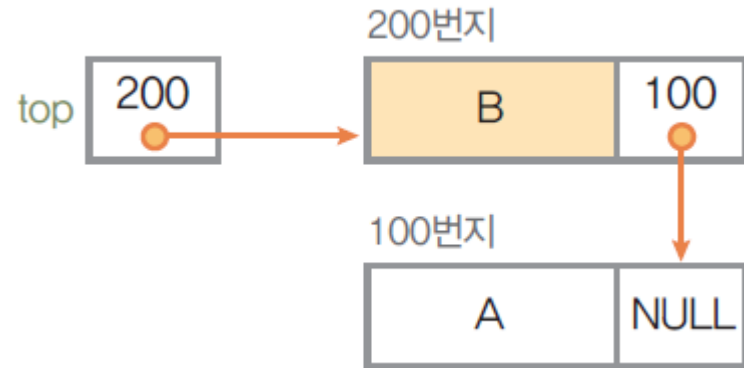
1 공백 스택 생성 : createStack(S); top NULL

2 원소 A 삽입 : push(S, A);

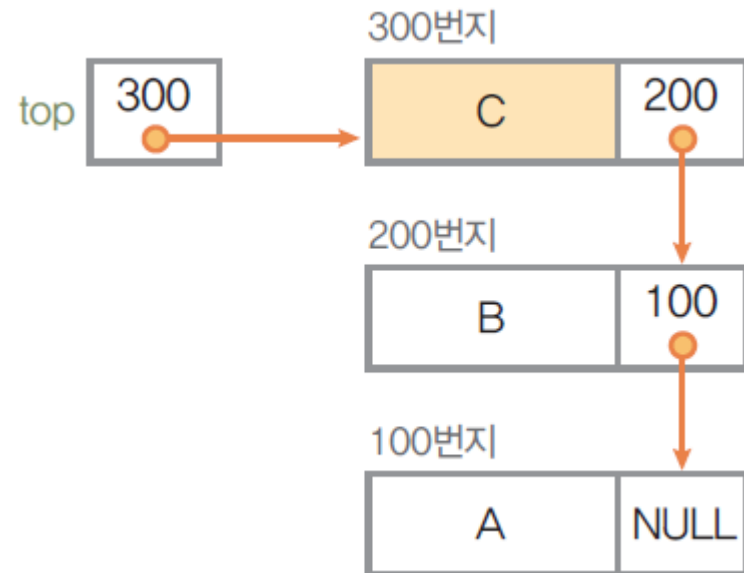


❖ 연결 리스트 스택의 push 연산

3 원소 B 삽입 : push(S, B);



4 원소 C 삽입 : push(S, C);

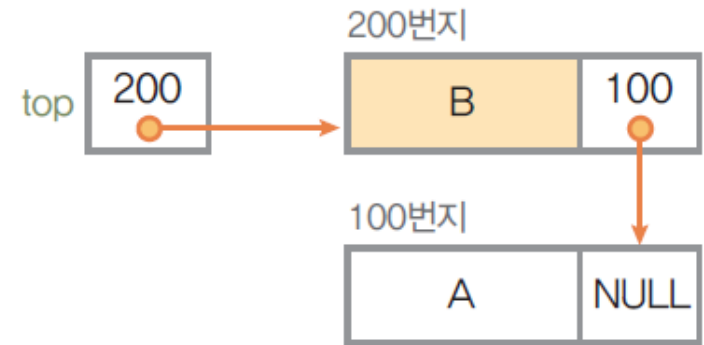


❖ 연결 리스트 스택의 pop 연산

```
// 스택의 top에서 원소를 삭제하는 연산
element pop()
{
    element item;
    stackNode* temp = top;

    if (top == NULL){
        printf("\n\n Stack is empty !\n");
        return 0;
    }
    else{
        item = temp->data;
        top = temp->link;
        free(temp);
        return item;
    }
}
```

5 원소 삭제 : pop(S);



❖ 연결 자료 구조를 이용한 스택 예제

```
#define _CRT_SECURE_NO_WARNINGS
#include "stdio.h"
#include "stdlib.h"
#include "string.h"

// 스택 원소(element)의 자료형을 int로 정의
typedef int element;

// 스택의 노드를 구조체로 정의
typedef struct stackNode {
    element data;
    struct stackNode *link;
} stackNode;

// 스택의 top 노드를 지정하기 위해 포인터 top 선언
stackNode* top;
```

❖ 연결 자료 구조를 이용한 스택 예제

```
// 스택이 공백 상태인지 확인하는 연산
int isEmpty()
{
    if (top == NULL)
        return 1;
    else
        return 0;
}

// 스택의 top에 원소를 삽입하는 연산
void push(element item)
{
    stackNode* temp = (stackNode *)malloc(sizeof(stackNode));
    temp->data = item;
    temp->link = top;        // 삽입 노드를 top의 위에 연결
    top = temp;             // top 위치를 삽입 노드로 이동
}
```


❖ 연결 자료 구조를 이용한 스택 예제

```
// 스택의 top에서 원소를 삭제하는 연산
element pop()
{
    element item;
    stackNode* temp = top;

    if (top == NULL)        // 스택이 공백 리스트인 경우
    {
        printf("\n\n Stack is empty !\n");
        return 0;
    }
    else                    // 스택이 공백 리스트가 아닌 경우
    {
        item = temp->data;
        top = temp->link;    // top 위치를 삭제 노드 아래로 이동
        free(temp);         // 삭제된 노드의 메모리 반환
        return item;        // 삭제된 원소 반환
    }
}
```

❖ 연결 자료 구조를 이용한 스택 예제

```
// 스택의 top 원소를 검색하는 연산
element peek()
{
    if (top == NULL)          // 스택이 공백 리스트인 경우
    {
        printf("\n\n Stack is empty !\n");
        return 0;
    }
    else                      // 스택이 공백 리스트가 아닌 경우
    {
        return(top->data);    // 현재 top의 원소 반환
    }
}
```

❖ 연결 자료 구조를 이용한 스택 예제

```
// 스택의 원소를 top에서 bottom 순서로 출력하는 연산
void printStack()
{
    stackNode* p = top;
    printf("\n STACK [ ");
    while (p)
    {
        printf("%d ", p->data);
        p = p->link;
    }
    printf("] ");
}
```

❖ 연결 자료 구조를 이용한 스택 예제

```
void main(void)
{
    element item;
    top = NULL;
    printf("\n** 연결 스택 연산 **\n");
    printStack();
    push(1);      // 1 삽입
    printStack();
    push(2);      // 2 삽입
    printStack();
    push(3);      // 3 삽입
    printStack();

    item = peek();
    printStack();           // 현재 top의 원소 출력
    printf("peek => %d", item);
```

❖ 연결 자료 구조를 이용한 스택 예제

```
    item = pop();           // 삭제
    printStack();
    printf("\t pop  => %d", item);

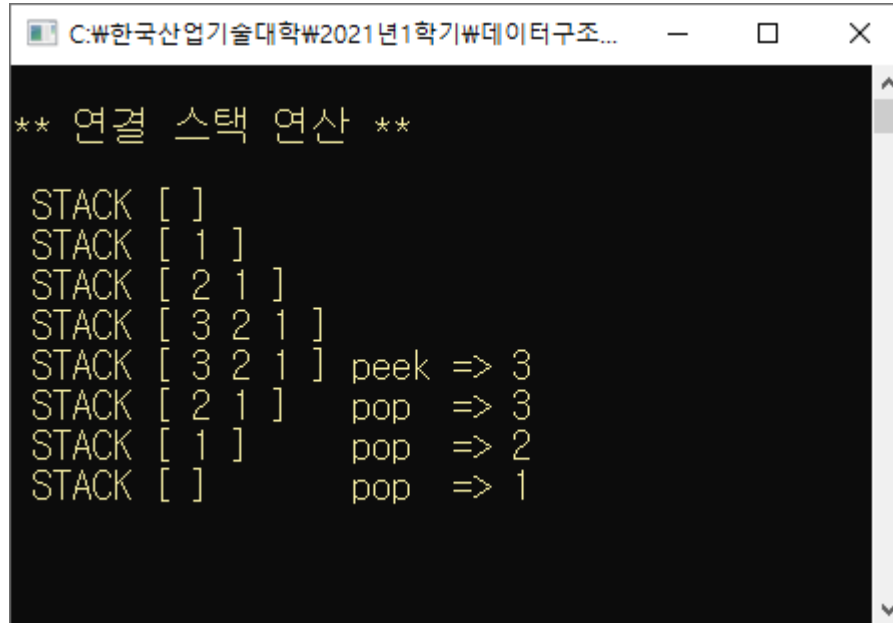
    item = pop();           // 삭제
    printStack();
    printf("\t pop  => %d", item);

    item = pop();           // 삭제
    printStack();
    printf("\t pop  => %d", item);

    getchar();
}
```

❖ 연결 자료 구조를 이용한 스택 예제 : [교재 243p](#)

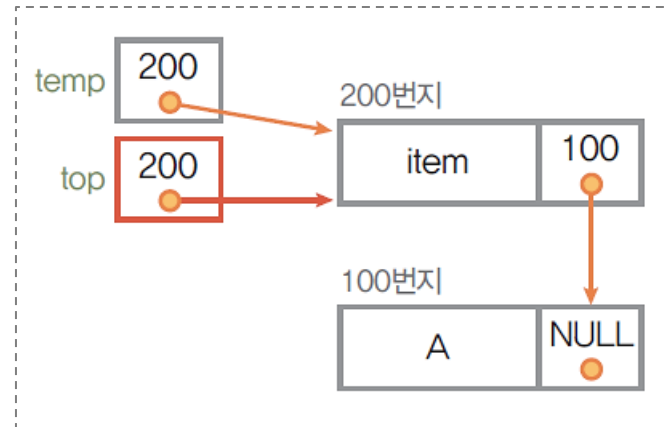
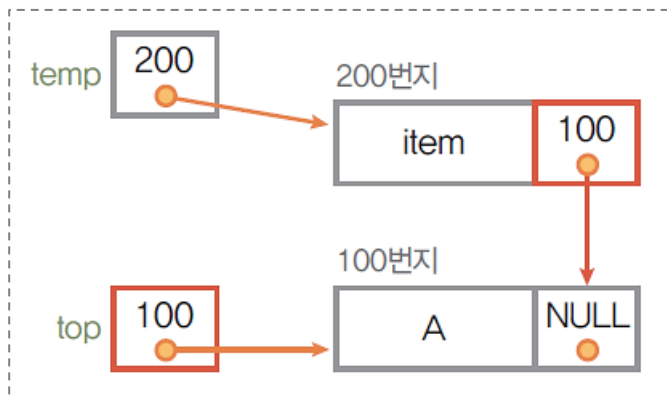
■ 실행 결과



```
** 연결 스택 연산 **  
STACK [ ]  
STACK [ 1 ]  
STACK [ 2 1 ]  
STACK [ 3 2 1 ]  
STACK [ 3 2 1 ] peek => 3  
STACK [ 2 1 ] pop => 3  
STACK [ 1 ] pop => 2  
STACK [ ] pop => 1
```

❖ 연결 스택의 삽입 연산 수행

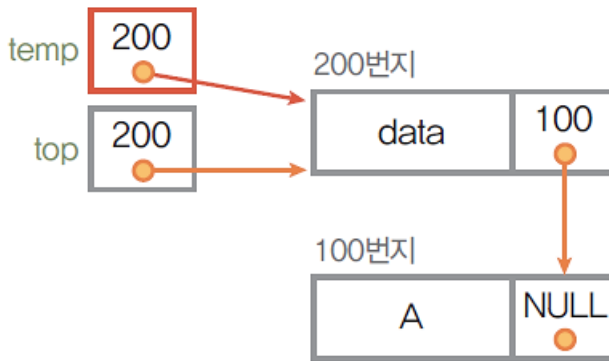
```
// 스택의 top에 원소를 삽입하는 연산
void push(element item)
{
    stackNode* temp = (stackNode *)malloc(sizeof(stackNode));
    temp->data = item;
    temp->link = top;           // 삽입 노드를 top의 위에 연결
    top = temp;                // top 위치를 삽입 노드로 이동
}
```



연결 자료를 이용한 스택의 구현

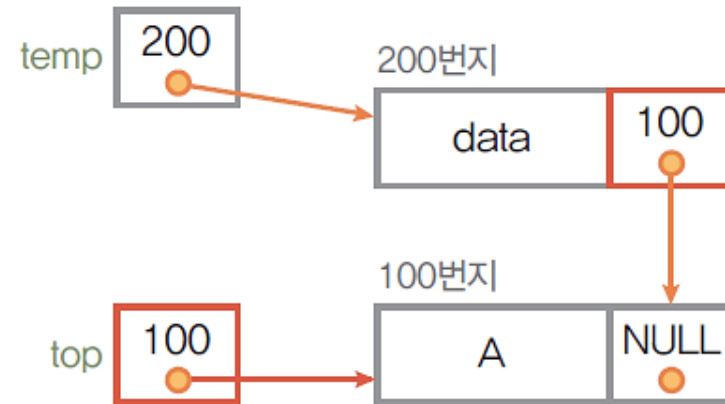
❖ 연결 스택의 삭제 연산 수행

```
stackNode* temp=top;
```



```
free(temp);
```

```
item = temp->data;  
top = temp->link;
```

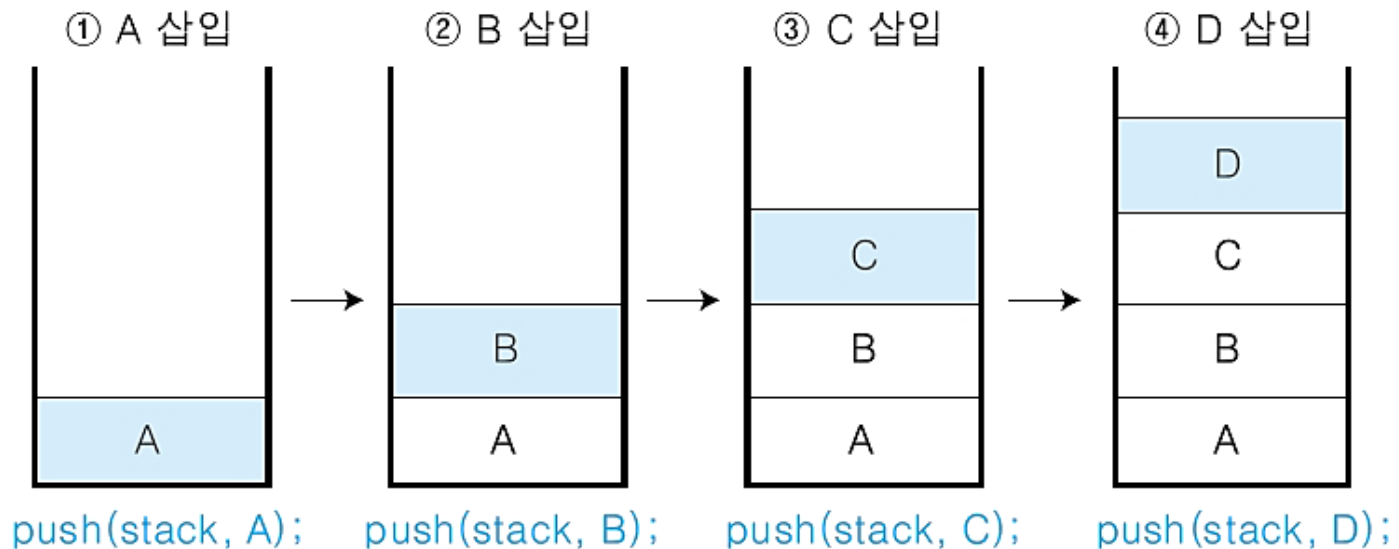


❖ 역순 문자열 만들기

- 스택의 후입 선출(LIFO)의 성질을 이용
- 문자열을 스택에 push 하기

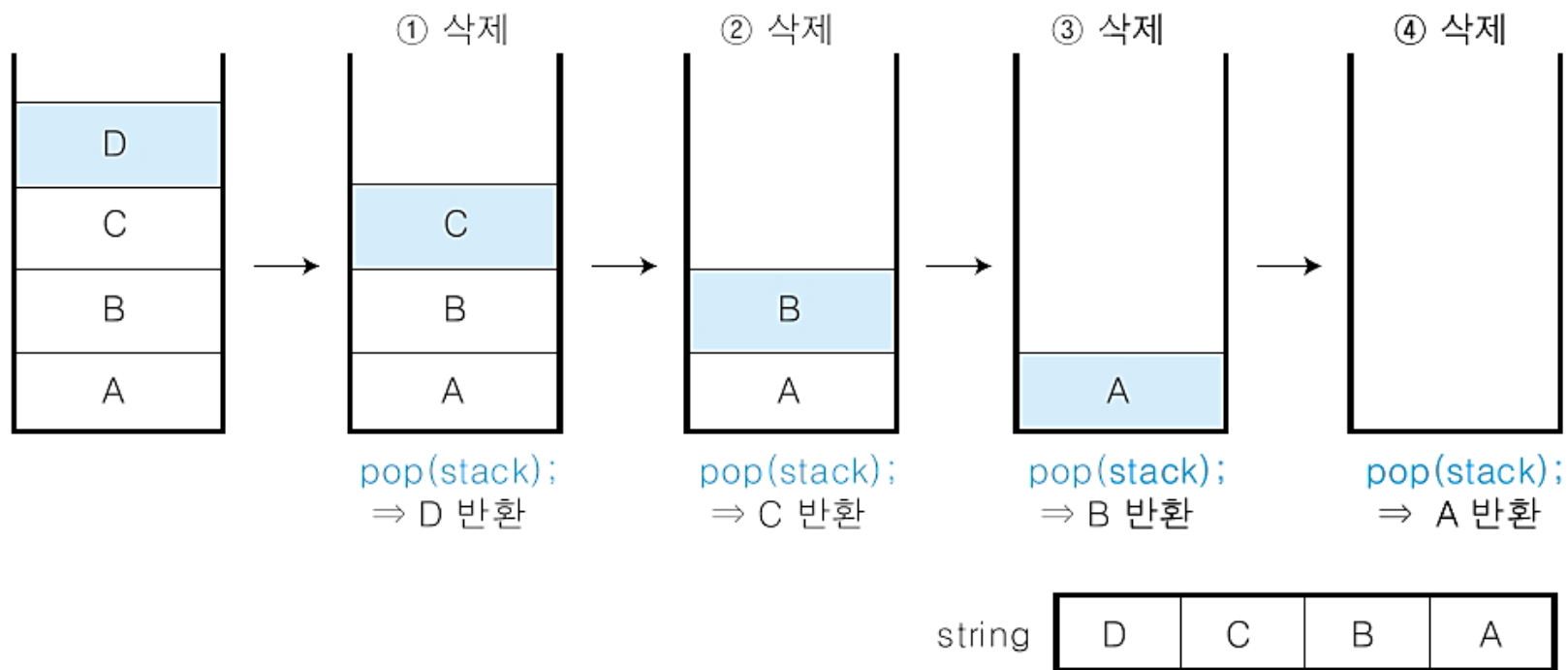
string

A	B	C	D
---	---	---	---



❖ 역순 문자열 만들기

- 스택을 pop 하여 문자열로 저장하기



❖ 역순 문자열 만들기 프로그램 (1/6)

```
#define _CRT_SECURE_NO_WARNINGS
#include "stdio.h"
#include "stdlib.h"
#include "string.h"

// 스택 원소(element)의 자료형을 char로 정의
typedef char element;

// 스택의 노드를 구조체로 정의
typedef struct stackNode {
    element data;
    struct stackNode *link;
} stackNode;

// 스택의 top 노드를 지정하기 위해 포인터 top 선언
stackNode* top;
```

❖ 역순 문자열 만들기 프로그램 (2/6)

```
// 스택의 top에 원소를 삽입하는 연산
void push(element item)
{
    stackNode* temp = (stackNode *)malloc(sizeof(stackNode));

    temp->data = item;
    temp->link = top;    // 삽입 노드를 top의 위에 연결
    top = temp;          // top 위치를 삽입 노드로 이동
}
```

❖ 역순 문자열 만들기 프로그램 (3/6)

```
// 스택의 top에서 원소를 삭제하는 연산
element pop()
{
    element item;
    stackNode* temp = top;

    if (top == NULL)        // 스택이 공백 리스트인 경우
    {
        printf("\n\n Stack is empty !\n");
        return 0;
    }
    else                    // 스택이 공백 리스트가 아닌 경우
    {
        item = temp->data;
        top = temp->link;    // top 위치를 삭제 노드 아래로 이동
        free(temp);         // 삭제된 노드의 메모리 반환
        return item;        // 삭제된 원소 반환
    }
}
```

❖ 역순 문자열 만들기 프로그램 (4/6)

```
// 스택의 원소를 top에서 bottom 순서로 출력하는 연산
void printStack()
{
    stackNode* p = top;
    printf("\n STACK [ ");
    while (p)
    {
        printf("%d ", p->data);
        p = p->link;
    }
    printf("] ");
}
```

❖ 역순 문자열 만들기 프로그램 (5/6)

```
void main()
{
    element item[50];
    int i = 0, k = 0;
    top = NULL;

    printf(" 문자열 입력 : ");
    scanf("%s", &item[i]);

    while(item[i] != NULL)
    {
        push(item[i]);
        i++;
    }
}
```

❖ 역순 문자열 만들기 프로그램 (6/6)

```
while(item[k] != NULL)
{
    stackNode* p = top;
    printStack();
    printf("\t pop => %c", p->data);
    pop();
    k++;
}
printStack();
printf("\n\n END\n");
}
```


❖ 역순 문자열 만들기 프로그램

■ 실행 결과

```
C:\₩한국산업기술대학₩2021년1학기₩데이터구조론₩실습예제₩스택₩Proj...  _  □  X
문자열 입력 : abcdef
STACK [ 102 101 100 99 98 97 ]      pop => f
STACK [ 101 100 99 98 97 ]      pop => e
STACK [ 100 99 98 97 ]      pop => d
STACK [ 99 98 97 ]      pop => c
STACK [ 98 97 ]      pop => b
STACK [ 97 ]      pop => a
STACK [ ]
END

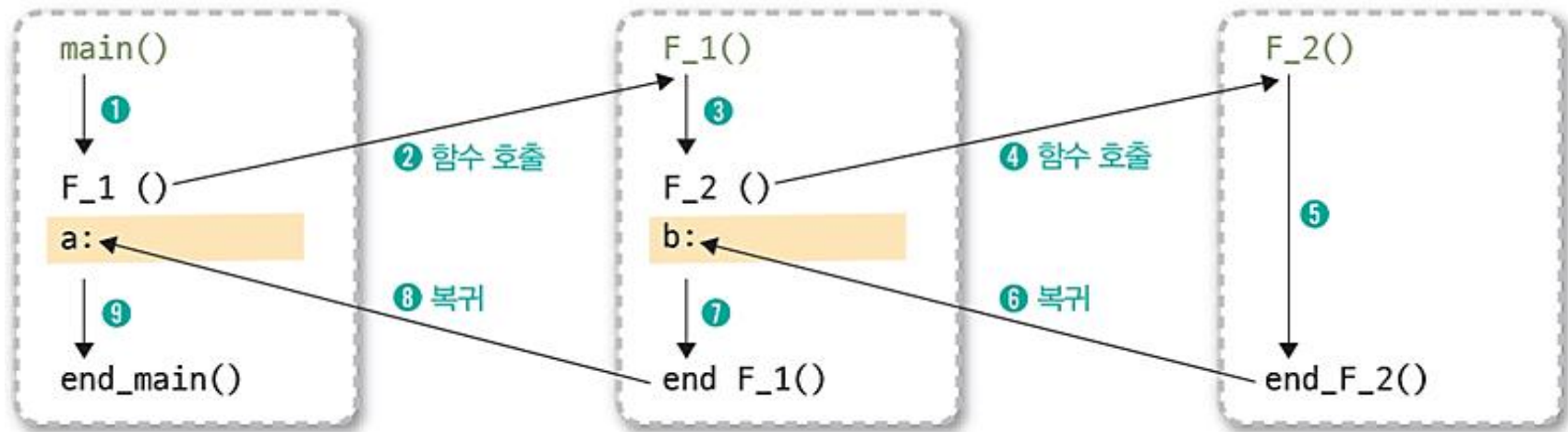
-----
Process exited after 9.295 seconds with return value 0
계속하려면 아무 키나 누르십시오 . . .
```

❖ 시스템 스택

- 프로그램에서의 호출과 복귀에 따른 수행 순서를 관리
 - 가장 가장 마지막에 호출된 함수가 가장 먼저 실행을 완료하고 복귀하는 후입 선출 구조이므로, 후입 선출 구조의 스택을 이용하여 수행순서 관리
 - 함수 호출이 발생하면 호출한 함수 수행에 필요한 지역변수, 매개변수 및 수행 후 복귀할 주소 등의 정보를 스택 프레임(stack frame)에 저장하여 시스템 스택에 삽입
 - 함수의 실행이 끝나면 시스템 스택의 top 원소(스택 프레임)를 삭제(pop)하면서 프레임에 저장되어있던 복귀주소를 확인하고 복귀
 - 함수 호출과 복귀에 따라 이 과정을 반복하여 전체 프로그램 수행이 종료되면 시스템 스택은 공백 스택이 됨

❖ 시스템 스택

- 함수의 호출과 복귀에 따른 전체 프로그램의 수행 순서



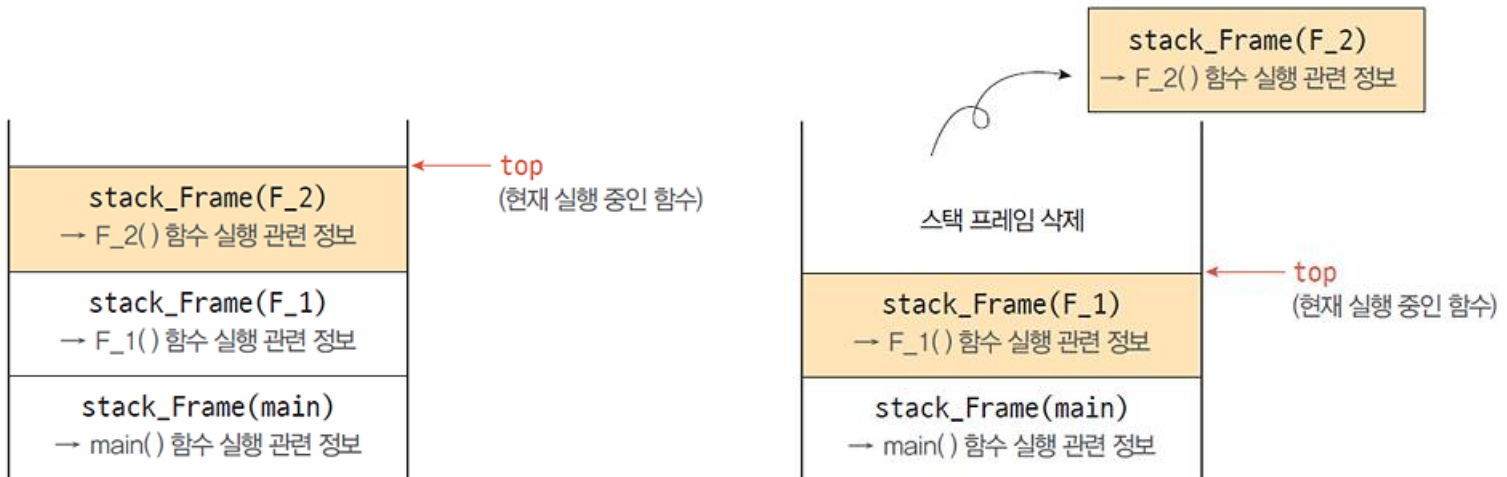
❖ 시스템 스택

- ① 프로그램이 실행을 시작하면 `main()` 함수가 실행되면서 `main()` 함수에 관련된 정보를 스택 프레임에 저장하여 시스템 스택에 삽입
- ② `main()` 함수 실행 중 `F_1()` 함수 호출을 만나면 함수 호출과 복귀에 필요한 정보를 스택 프레임에 저장하여 시스템 스택에 삽입하고 호출된 함수인 `F_1()` 함수로 이동
 - ✓ 스택 프레임에는 호출된 함수의 수행이 끝나고 `main()` 함수로 복귀할 주소 `a`를 저장



❖ 시스템 스택

- ③ 호출된 함수 F_1() 함수 실행
- ④ F_1() 함수 실행 중에 F_2() 함수를 만나면 다시 함수 호출과 복귀에 필요한 정보를 스택 프레임에 저장하여 시스템 스택에 삽입하고, 호출된 함수인 F_2() 함수를 실행. 스택 프레임에는 F_1() 함수로 복귀할 주소 b를 저장
- ⑤ 호출된 함수 F_2() 함수를 실행 완료
- ⑥ F_2() 함수 실행이 끝나면 F_2() 함수를 호출했던 이전 위치로 복귀하여 이전 함수 F_1()의 작업을 계속 함



❖ 시스템 스택

- ⑦ F_1() 함수로 복귀하여 F_1() 함수의 나머지 부분 실행
- ⑧ F_1() 함수 실행 종료, main() 함수로 복귀 : 스택의 top에 있는 스택 프레임을 pop하여 정보를 확인하고 복귀 및 작업 전환을 실행
- ⑨ main() 함수 실행 완료(전체 프로그램 실행 완료) : 정상적인 함수 호출과 복귀가 모두 완료되었으므로 시스템 스택은 공백이 됨



❖ 수식의 괄호 검사

- 수식에 포함되어있는 괄호는 가장 마지막에 열린 괄호를 가장 먼저 닫아 주어야 하는 후입 선출 구조로 구성되어있으므로, 후입 선출 구조의 스택을 이용하여 괄호를 검사한다.
- 수식을 왼쪽에서 오른쪽으로 하나씩 읽으면서 괄호 검사
 - 왼쪽 괄호를 만나면 스택에 push
 - 오른쪽 괄호를 만나면 스택을 pop하여 마지막에 저장한 괄호와 같은 종류인지를 확인
 - ☞ 같은 종류의 괄호가 아닌 경우 괄호의 짝이 잘못 사용된 수식!
- 수식에 대한 검사가 모두 끝났을 때 스택은 공백 스택
 - 수식이 끝났어도 스택이 공백이 되지 않으면 괄호의 개수가 틀린 수식!

❖ 수식의 괄호 검사 알고리즘

알고리즘

수식의 괄호 쌍 검사

```
testPair( )
  exp ← Expression;
  Stack ← null;
  while (true) do{
    symbol ← getSymbol(exp);
    case {
      symbol = "(" or "[" or "{" :
        push(Stack, symbol);
      symbol = ")" :
        open_pair ← pop(Stack);
        if (open_pair ≠ "(") then return false;
      symbol = "]" :
        open_pair ← pop(Stack);
        if (open_pair ≠ "[") then return false;
      symbol = "}" :
        open_pair ← pop(Stack);
        if (open_pair ≠ "{") then return false;
      symbol = null :
        if (isEmpty(Stack)) then return true;
        else return false;
    }
  }
end testPair( )
```


❖ 수식의 괄호 검사 예

$\{(A+B)-3\} * 5 + [\{\cos(x+y)+7\}-1] * 4$

$\{(A+B)-3\} * 5 + [\{\cos(x+y)+7\}-1] * 4$
① push(stack, {)
② push(stack, ()



스택 stack

$\{(A+B)-3\} * 5 + [\{\cos(x+y)+7\}-1] * 4$
③ pop(stack)

같은 종류인지 확인

삭제된 원소 : (



스택 stack

❖ 수식의 괄호 검사 예

$\{ (A + B) - 3 \} * 5 + [\{ \cos(x + y) + 7 \} - 1] * 4$

4 pop(stack)

같은 종류인지 확인

삭제된 원소 : {



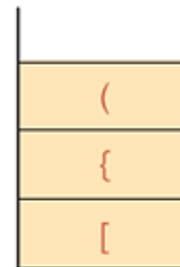
스택 stack

$\{ (A + B) - 3 \} * 5 + [\{ \cos(x + y) + 7 \} - 1] * 4$

7 push(stack, ()

6 push(stack, {)

5 push(stack, [)



스택 stack

$\{ (A + B) - 3 \} * 5 + [\{ \cos(x + y) + 7 \} - 1] * 4$

8 pop(stack)

같은 종류인지 확인

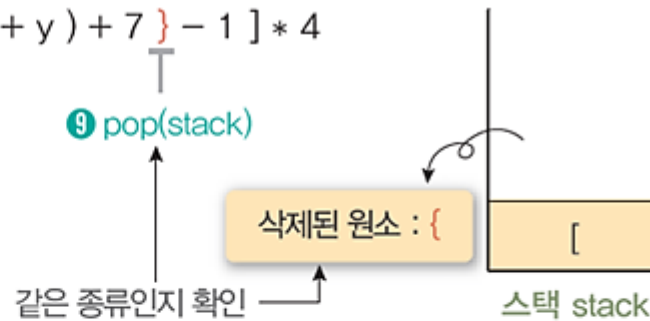
삭제된 원소 : (



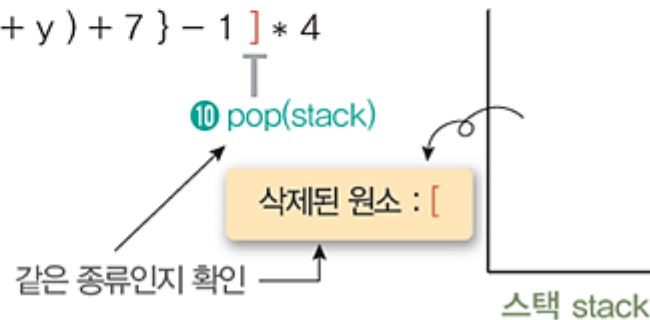
스택 stack

❖ 수식의 괄호 검사 예

$\{ (A + B) - 3 \} * 5 + [\{ \cos (x + y) + 7 \} - 1] * 4$



$\{ (A + B) - 3 \} * 5 + [\{ \cos (x + y) + 7 \} - 1] * 4$



❖ 수식의 괄호 검사 프로그램 (1/7)

```
#define _CRT_SECURE_NO_WARNINGS
#include "stdio.h"
#include "stdlib.h"
#include "string.h"

// 스택 요소(element)의 자료형을 char로 정의
typedef char element;

// 스택의 노드를 구조체로 정의
typedef struct stackNode {
    element data;
    struct stackNode *link;
} stackNode;

// 스택의 top 노드를 지정하기 위해 포인터 top 선언
stackNode* top;
```

❖ 수식의 괄호 검사 프로그램 (2/7)

```
// 스택이 공백 상태인지 확인하는 연산
int isEmpty()
{
    if (top == NULL)
        return 1;
    else
        return 0;
}

// 스택의 top에 원소를 삽입하는 연산
void push(element item)
{
    stackNode* temp = (stackNode *)malloc(sizeof(stackNode));
    temp->data = item;
    temp->link = top;        // 삽입 노드를 top의 위에 연결
    top = temp;             // top 위치를 삽입 노드로 이동
}
```

❖ 수식의 괄호 검사 프로그램 (3/7)

```
// 스택의 top에서 원소를 삭제하는 연산
element pop()
{
    element item;
    stackNode* temp = top;

    if (top == NULL)        // 스택이 공백 리스트인 경우
    {
        printf("\n\n Stack is empty !\n");
        return 0;
    }
    else                    // 스택이 공백 리스트가 아닌 경우
    {
        item = temp->data;
        top = temp->link;    // top 위치를 삭제 노드 아래로 이동
        free(temp);         // 삭제된 노드의 메모리 반환
        return item;        // 삭제된 원소 반환
    }
}
```

❖ 수식의 괄호 검사 프로그램 (4/7)

```
// 수식의 괄호를 검사하는 연산
int testPair(char *exp)
{
    char symbol, open_pair;
    // char형 포인터 매개변수로 받은 수식 exp의
    // 길이를 계산하여 length 변수에 저장
    int i, length = strlen(exp);
    top = NULL;
    for (i = 0; i < length; i++)
    {
        symbol = exp[i];
        switch (symbol)
        {
            // (1) 왼쪽 괄호는 스택에 삽입
            case '(':
            case '[':
            case '{':
                push(symbol);
                break;
        }
    }
}
```

❖ 수식의 괄호 검사 프로그램 (5/7)

```
// (2) 오른쪽 괄호를 읽으면,  
case ')':  
case ']':  
case '}':  
    if (top == NULL)  
        return 0;  
    else  
    {  
        // (2)-1 스택에서 마지막으로 읽은 괄호를 꺼냄  
        open_pair = pop();  
        // (2)-2 괄호 쌍이 맞는지 검사  
        if ((open_pair == '(' && symbol != ')') ||  
            (open_pair == '[' && symbol != ']') ||  
            (open_pair == '{' && symbol != '}'))  
            return 0; // (2)-3 괄호 쌍이 틀리면 수식 오류  
        // (2)-4 괄호 쌍이 맞으면 다음 symbol 검사를 계속함  
        else  
            break;  
    }  
}
```


❖ 수식의 괄호 검사 프로그램 (6/7)

```
}  
if (top == NULL)  
    return 1;          // 수식 검사를 마친 후 스택이 공백이면 1을 반환  
else  
    return 0;          // 공백이 아니면 0을 반환(수식 괄호 틀림)  
}
```

❖ 수식의 괄호 검사 프로그램 (7/7)

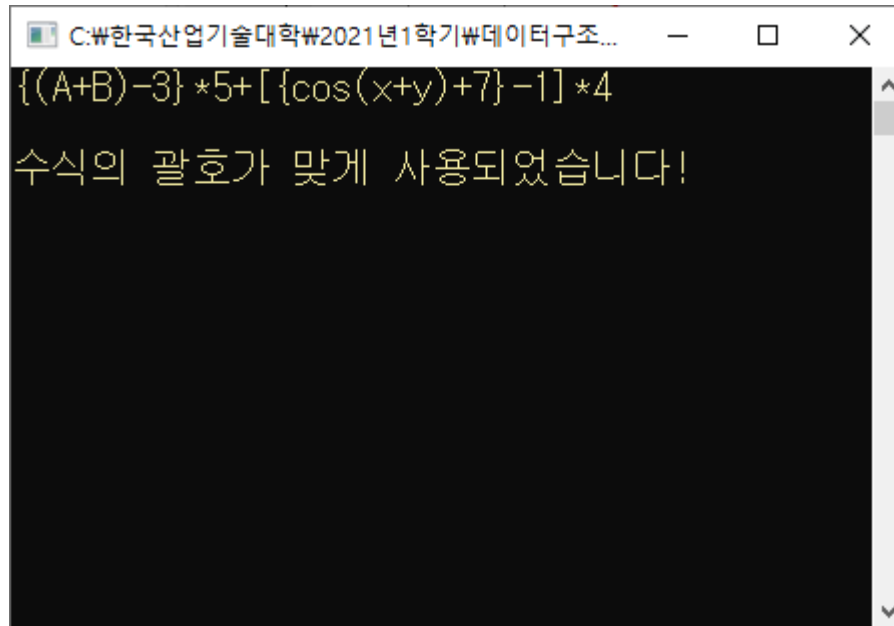
```
void main(void)
{
    char* express = "{(A+B)-3}*5+[{cos(x+y)+7}-1]*4";
    printf("%s", express);

    if (testPair(express) == 1)
        printf("\n\n수식의 괄호가 맞게 사용되었습니다!");
    else
        printf("\n\n수식의 괄호가 틀렸습니다!");

    getchar();
}
```

❖ 수식의 괄호 검사 프로그램 : 교재 253p

■ 실행 결과



A screenshot of a Windows-style application window. The title bar text is "C:\₩한국산업기술대학₩2021년1학기₩데이터구조...". The window has a black background with yellow text. The first line of text is the mathematical expression $\{(A+B)-3\} * 5 + [\{\cos(x+y)+7\}-1] * 4$. The second line of text is "수식의 괄호가 맞게 사용되었습니다!".

```
C:\₩한국산업기술대학₩2021년1학기₩데이터구조...  
{(A+B)-3} * 5 + [ {cos(x+y)+7} -1] * 4  
수식의 괄호가 맞게 사용되었습니다!
```

❖ 수식의 후위 표기법 변환

- 전위 표기법(prefix notation)
 - 연산자를 피연산자 앞에 표기하는 방법
예) $+AB$
- 중위 표기법(infix notation)
 - 연산자를 피 연산자의 가운데 표기하는 방법
예) $A+B$
- 후위 표기법(postfix notation)
 - 연산자를 피 연산자 뒤에 표기하는 방법
예) $AB+$

❖ 중위 표기식의 전위 표기식 변환

예) $A * B - C / D$

- ① 수식의 각 연산자에 대해서 우선 순위에 따라 괄호를 사용하여 다시 표현한다.

$((A * B) - (C / D))$

- ② 각 연산자를 그에 대응하는 왼쪽괄호의 앞으로 이동 시킨다.
 $-(* (A \ B) / (C \ D))$

- ③ 괄호를 제거한다.

$- * AB / CD$

❖ 중위 표기식의 후위 표기식 변환 방법

예) $A * B - C / D$

- ① 수식의 각 연산자에 대해서 우선 순위에 따라 괄호를 사용하여 다시 표현한다.

$((A * B) - (C / D))$

- ② 각 연산자를 그에 대응하는 오른쪽괄호의 뒤로 이동 시킨다.

$((A B) * (C D) /) -$

- ③ 괄호를 제거한다.

$AB * CD / -$

❖ 스택을 사용하여 입력된 중위 표기식을 후위 표기식으로 변환

■ 변환 방법

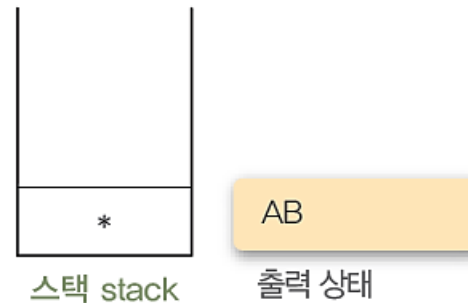
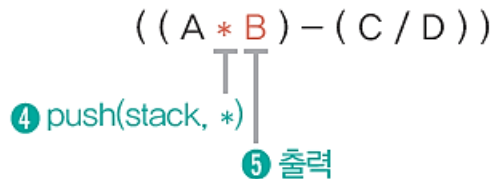
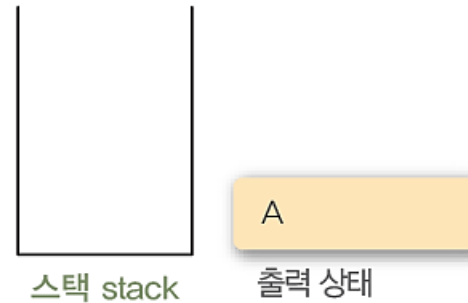
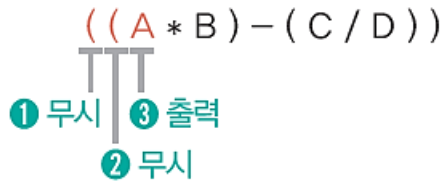
- ① 왼쪽괄호를 만나면 무시하고 다음 문자를 읽는다.
- ② 피연산자를 만나면 출력한다.
- ③ 연산자를 만나면 스택에 push한다.
- ④ 오른쪽 괄호를 만나면 스택을 pop하여 출력한다.
- ⑤ 수식이 끝나면 스택이 공백이 될 때까지 pop하여 출력한다.

❖ 스택을 이용한 수식의 후위 표기법 변환

알고리즘	후위 표기법으로 변환
<pre>infix_to_postfix(exp) while (true) do { symbol ← getSymbol(exp); case { symbol = operand : // 피연산자 처리 print(symbol); symbol = operator : // 연산자 처리 push(stack, symbol); symbol = ")" : // 오른쪽괄호 처리 print(pop(Stack)); symbol = null : // 수식의 끝 처리 while (top > -1) do print(pop(Stack)); else : } } } end infix_to_postfix()</pre>	

❖ 스택을 이용한 수식의 후위 표기법 변환

- 스택을 이용하여 수식 $A*B-C/D$ 를 후위 표기법으로 변환

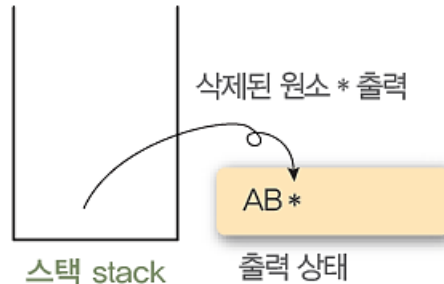


❖ 스택을 이용한 수식의 후위 표기법 변환

- 스택을 이용하여 수식 $A*B-C/D$ 를 후위 표기법으로 변환

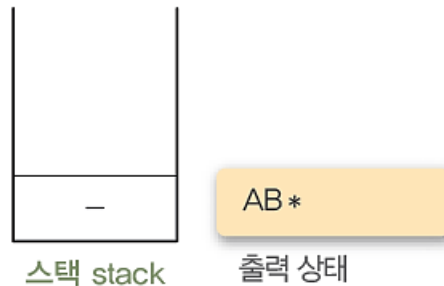
$((A * B) - (C / D))$

6 pop(stack)



$((A * B) - (C / D))$

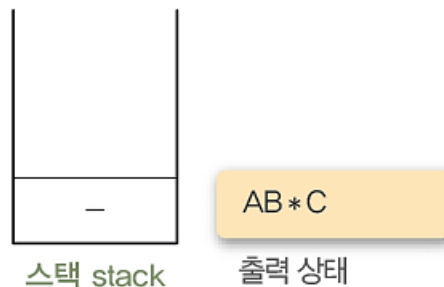
7 push(stack, -)



$((A * B) - (C / D))$

8 무시

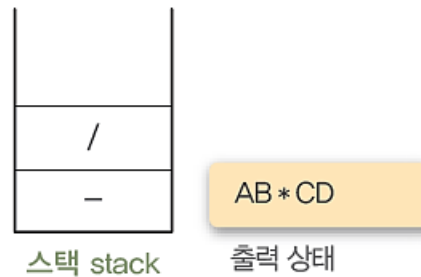
9 출력



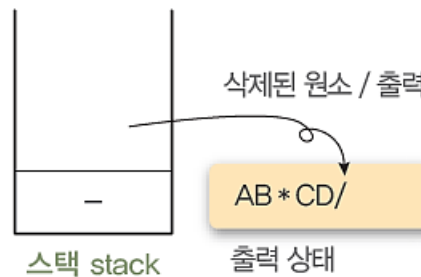
❖ 스택을 이용한 수식의 후위 표기법 변환

- 스택을 이용하여 수식 $A*B-C/D$ 를 후위 표기법으로 변환

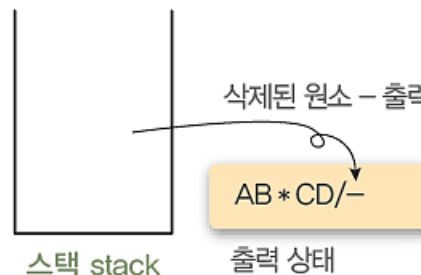
$((A * B) - (C / D))$
 ⑩ push(stack, /)
 ⑪ 출력



$((A * B) - (C / D))$
 ⑫ pop(stack)



$((A * B) - (C / D))$
 ⑬ pop(stack)



❖ 스택을 이용한 후위 표기법 수식의 연산

■ 연산 방법

- ① 피연산자를 만나면 스택에 push 한다.
- ② 연산자를 만나면 필요한 만큼의 피연산자를 스택에서 pop하여 연산하고, 연산결과를 다시 스택에 push 한다.
- ③ 수식이 끝나면, 마지막으로 스택을 pop하여 출력한다.

- 수식이 끝나고 스택에 마지막으로 남아있는 원소는 전체 수식의 연산결과 값이 된다.

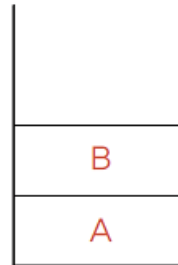
❖ 스택을 이용한 후위 표기법 수식의 연산

알고리즘	후위 표기법으로 수식 연산
<pre>evalPostfix(exp) while (true) do { symbol ← getSymbol(exp); case { symbol = operand : // 피연산자 처리 push(stack, symbol); symbol = operator : // 연산자 처리 opr2 ← pop(stack)); opr1 ← pop(stack)); result ← opr1 op(symbol) opr2; // 스택에서 꺼낸 피연산자들을 연산자로 연산 push(stack, result); symbol = null : // 후위수식의 끝 print(pop(stack)); } } end evalPostfix()</pre>	

❖ 스택을 이용한 후위 표기법 수식의 연산

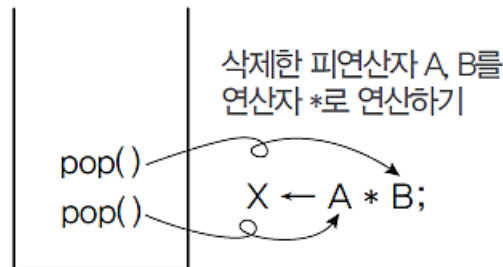
- 위의 알고리즘으로 수식 $AB*CD/-$ 를 연산

$A \ B \ * \ C \ D \ / \ -$
 ① $push(stack, A)$
 ② $push(stack, B)$



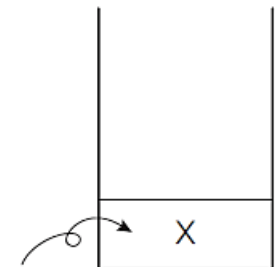
스택 stack

$A \ B \ * \ C \ D \ / \ -$
 ③ $pop(stack)$
 $pop(stack)$



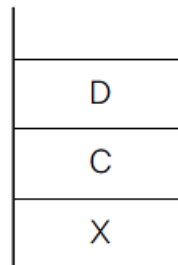
스택 stack

연산 결과 X를 다시 스택에 삽입
 $push(stack, X);$



스택 stack

$A \ B \ * \ C \ D \ / \ -$
 ④ $push(stack, C)$
 ⑤ $push(stack, D)$



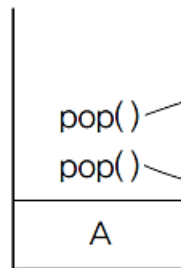
스택 stack

❖ 스택을 이용한 후위 표기법 수식의 연산

- 위의 알고리즘으로 수식 $AB*CD/-$ 를 연산

A B * C D / -

⑥ pop(stack)
pop(stack)

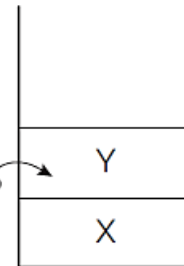


스택 stack

삭제한 피연산자 C, D를
연산자 /로 연산하기

$Y \leftarrow C / D;$

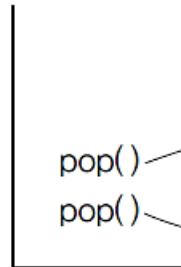
연산 결과 Y를 다시 스택에 삽입
push(stack, Y);



스택 stack

A B * C D / -

⑦ pop(stack)
pop(stack)

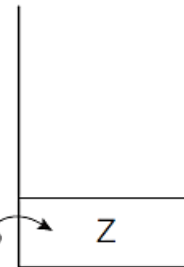


스택 stack

삭제한 피연산자 X, Y를
연산자 -로 연산하기

$Z \leftarrow X - Y;$

연산 결과 Z를 다시 스택에 삽입
push(stack, Z);



스택 stack

❖ 수식을 후위 표기법으로 연산하기 프로그램 (1/6)

```
#define _CRT_SECURE_NO_WARNINGS
#include "stdio.h"
#include "stdlib.h"
#include "string.h"

// 스택 원소(element)의 자료형을 int로 정의
typedef int element;

// 스택의 노드를 구조체로 정의
typedef struct stackNode {
    element data;
    struct stackNode *link;
} stackNode;

// 스택의 top 노드를 지정하기 위해 포인터 top 선언
stackNode* top;
```


❖ 수식을 후위 표기법으로 연산하기 프로그램 (2/6)

```
// 스택이 공백 상태인지 확인하는 연산
int isEmpty()
{
    if (top == NULL)
        return 1;
    else
        return 0;
}

// 스택의 top에 원소를 삽입하는 연산
void push(element item)
{
    stackNode* temp = (stackNode *)malloc(sizeof(stackNode));
    temp->data = item;
    temp->link = top;        // 삽입 노드를 top의 위에 연결
    top = temp;             // top 위치를 삽입 노드로 이동
}
```

❖ 수식을 후위 표기법으로 연산하기 프로그램 (3/6)

```
// 스택의 top에서 원소를 삭제하는 연산
element pop()
{
    element item;
    stackNode* temp = top;

    if (top == NULL)           // 스택이 공백 리스트인 경우
    {
        printf("\n\n Stack is empty !\n");
        return 0;
    }
    else                       // 스택이 공백 리스트가 아닌 경우
    {
        item = temp->data;
        top = temp->link;      // top 위치를 삭제 노드 아래로 이동
        free(temp);           // 삭제된 노드의 메모리 반환
        return item;          // 삭제된 원소 반환
    }
}
```

❖ 수식을 후위 표기법으로 연산하기 프로그램 (4/6)

```
// 후위 표기법 수식을 계산하는 연산
element evalPostfix(char *exp)
{
    int opr1, opr2, value, i = 0;
    // char형 포인터 매개변수로 받은 수식 exp의 길이를 계산하여
    // length 변수에 저장
    int length = strlen(exp);
    char symbol;
    top = NULL;

    for (i = 0; i < length; i++)
    {
        symbol = exp[i];
        if (symbol != '+' && symbol != '-' && symbol != '*' && symbol != '/')
        {
            value = symbol - '0';
            push(value);
        }
        else
        {
            opr2 = pop();
            opr1 = pop();
```

❖ 수식을 후위 표기법으로 연산하기 프로그램 (5/6)

```
// 변수 opr1과 opr2에 대해 symbol에 저장된 연산자를 연산
switch (symbol)
{
case '+':
    push(opr1 + opr2);
    break;
case '-':
    push(opr1 - opr2);
    break;
case '*':
    push(opr1 * opr2);
    break;
case '/':
    push(opr1 / opr2);
    break;
}
}
// 수식 exp에 대한 처리를 마친 후 스택에 남아 있는 결과값을 pop하여 반환
return pop();
}
```

❖ 수식을 후위 표기법으로 연산하기 프로그램 (6/6)

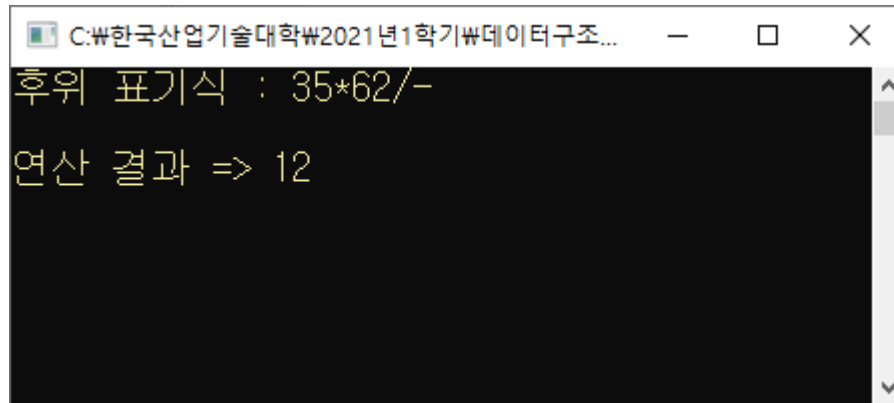
```
void main(void)
{
    int result;
    char* express = "35*62/-";
    printf("후위 표기식 : %s", express);

    result = evalPostfix(express);
    printf("\n\n연산 결과 => %d", result);

    getchar();
}
```

❖ 수식을 후위 표기법으로 연산하기 프로그램 : 교재 260p

- 위의 알고리즘으로 수식 $AB*CD/-$ 를 연산



```
C:\₩한국산업기술대학₩2021년1학기₩데이터구조...
후위 표기식 : 35*62/-
연산 결과 => 12
```